

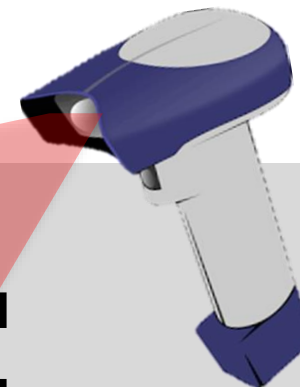


20231105

이미지처리팀



ICML, 2023
Data-centric ML Workshop



ObjectLab:

Automated Diagnosis
of Mislabeled Images
in Object Detection Data



강인하 | 김준철 | 최승준 | 김현진 | 허정원



INTRO: Data-centric AI

- **과거 연구:** specific한 task로 '모델'을 학습하고 성능을 평가
- **Data-centric AI:** 어떤 '데이터'로 모델을 학습시켰을 때 성능이 향상될 수 있었으며 무엇이 '좋은 데이터'인지를 평가

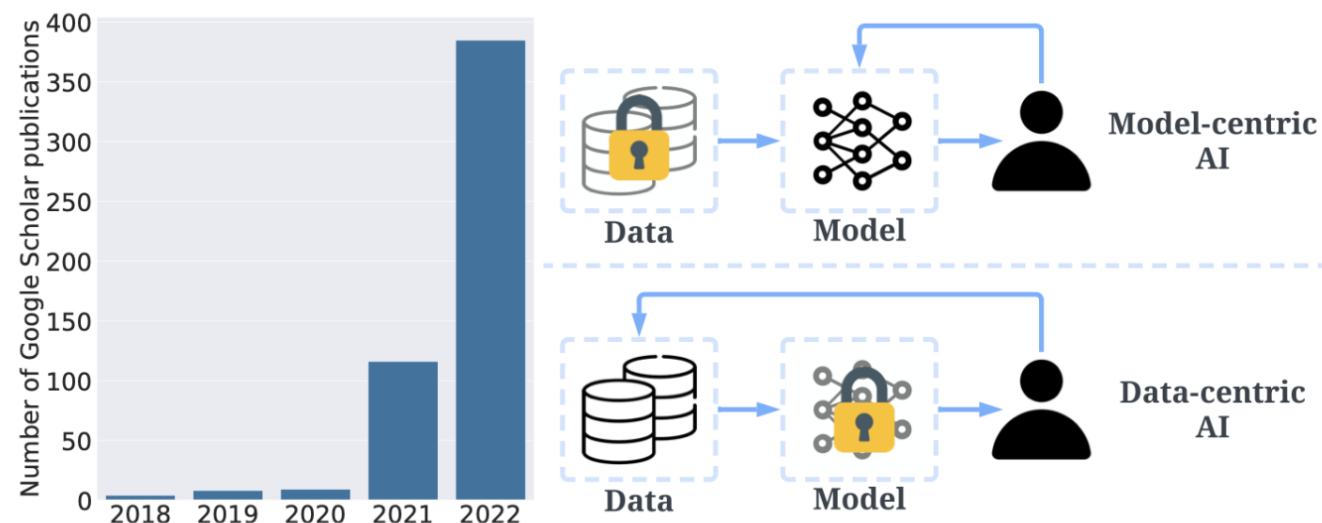


Figure 1: **Left:** Tendency of DCAI over the past five years. The statistics are collected by querying Google Scholar with the exactly matched phrase “data-centric AI”. **Right:** Model-centric AI versus DCAI.

INTRO: Data-centric AI

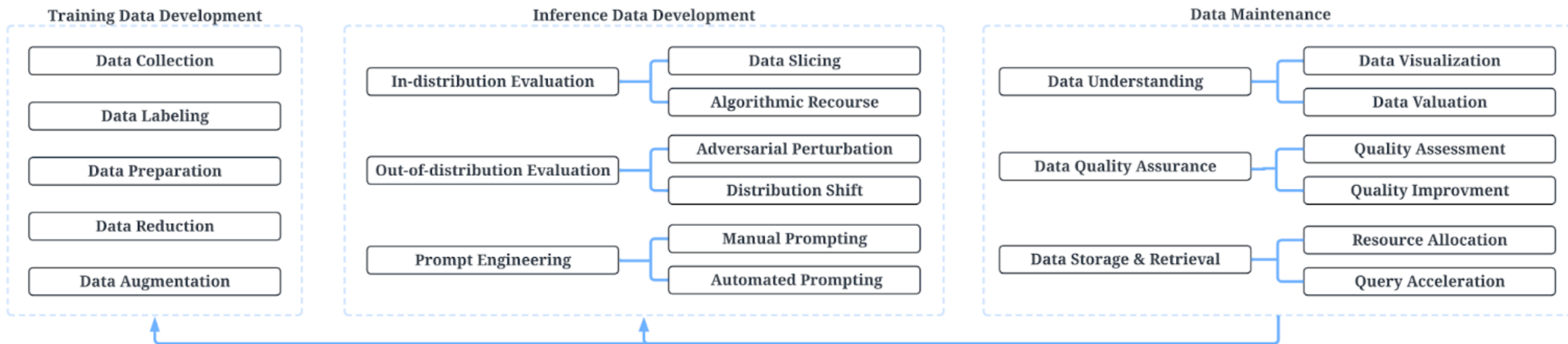


Figure 2: A big picture of DCAI missions and their representative tasks/subtasks.

INTRO: Data-centric AI

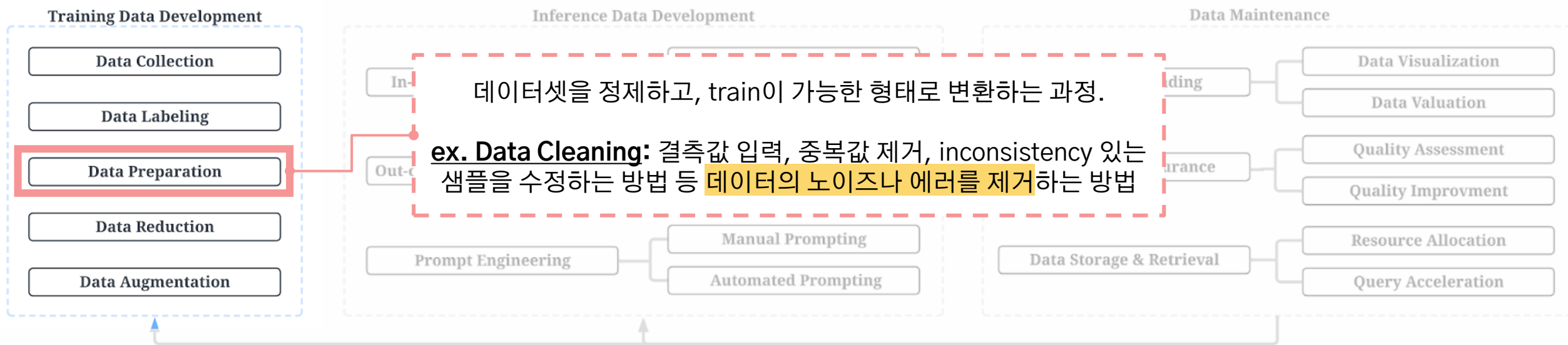
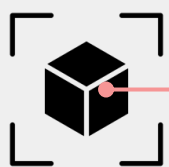
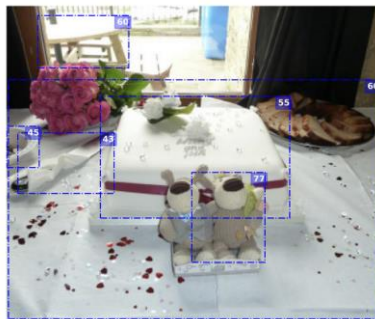
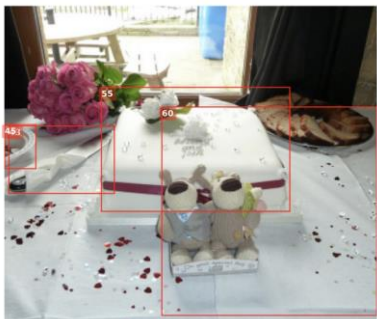


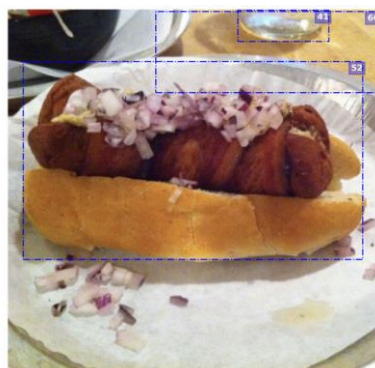
Figure 2: A big picture of DCAI missions and their representative tasks/subtasks.



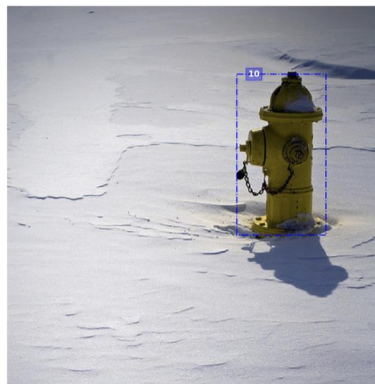
INTRO: Problem Statements



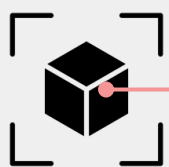
Badly Located Error



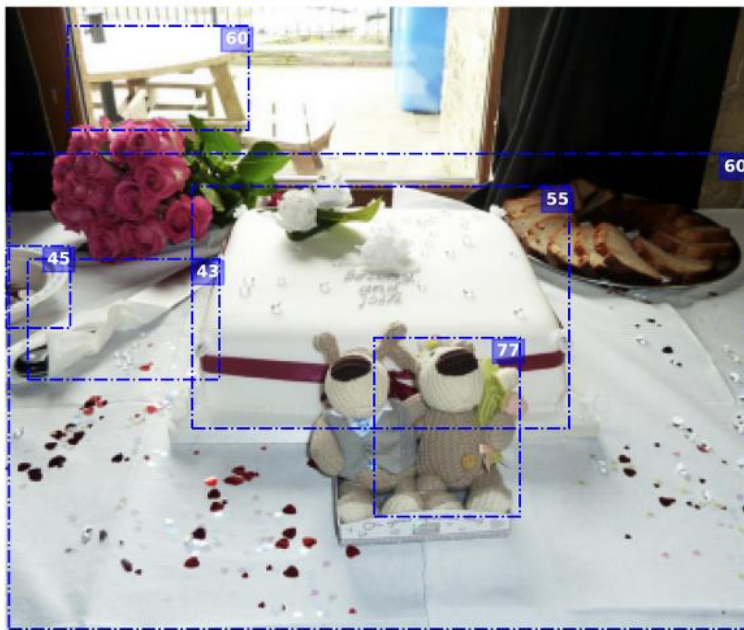
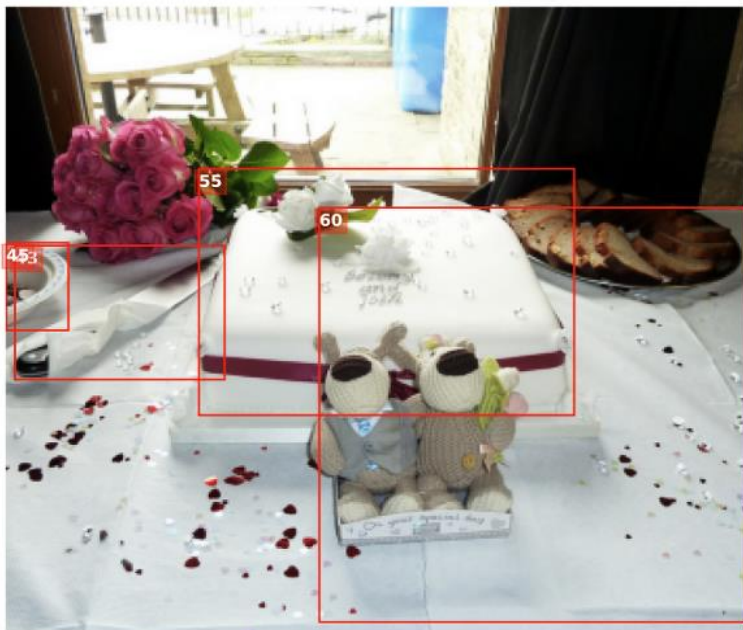
Swapped Error



Overlooked Error



INTRO: Problem Statements

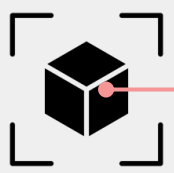


Badly Located Error

annotators poorly outlined only half of the dining table(class #60) which the model localized much better (with confidence 0.964), leading to a low Badly-Located score in ObjectLab.



- GT의 bbox가 object 전체를 포함하고 있지 않거나 위치가 정확하지 않은 경우
- 60번 class(=table)를 보면 prediction 결과에서는 table 전체가 bbox에 포함되지만 GT에서는 테이블의 일부만 포함 됨.

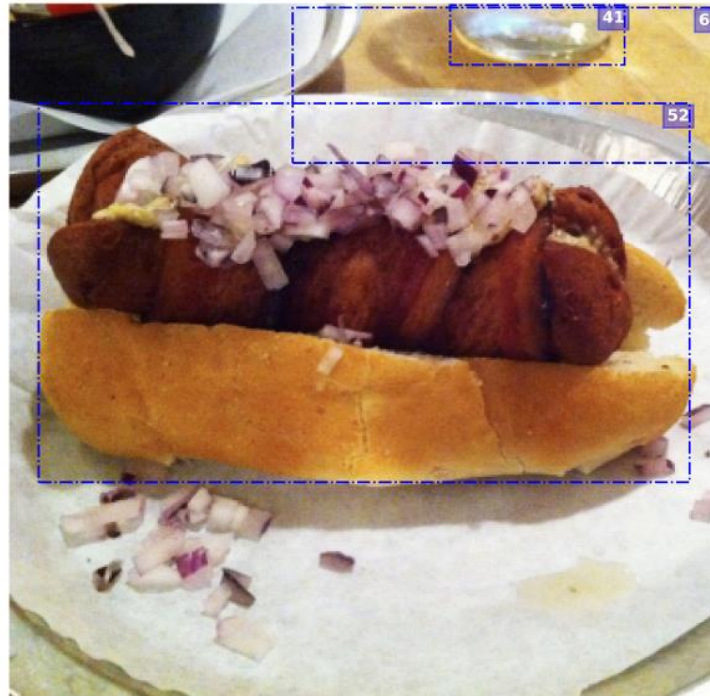


INTRO: Problem Statements



Badly Located Error

: GT의 bbox가 object 전체를 포함하고 있지 않거나 위치가 정확하지 않은 경우



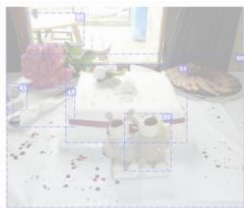
Swapped Error

the glass object on the right is incorrectly annotated as a bowl(class #45), while the model predicted cup(class #41) with confidence 0.962, leading to a low Swapped-score in ObjectLab.

- GT의 bbox의 위치는 맞지만, 그 클래스가 틀린 경우를 말한다.
- GT의 빨간색 bbox는 상단의 물잔을 bowl에 해당하는 45번 class로 표기한 반면, ObjectLab으로 교정한 결과 cup에 해당하는 41번 class로 옳게 바뀐.

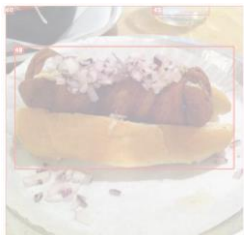


INTRO: Problem Statements



Badly Located Error

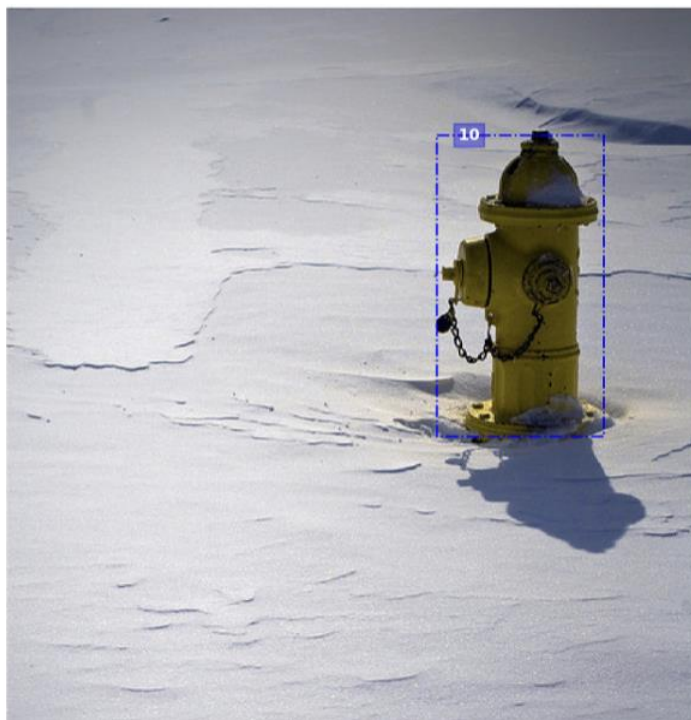
: GT의 bbox가 object 전체를 포함하고 있지 않거나 위치가 정확하지 않은 경우



Swapped Error

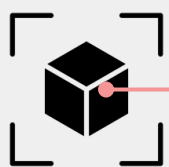
: GT의 bbox 위치는 맞지만, Class가 틀린 경우

- GT에 존재해야 할 bbox가 missing된 경우
- 왼쪽의 GT bbox에는 소화전에 bbox가 존재하지 않지만, ObjectLab의 결과에서는 소화전에 올바른 bbox가 존재함.



Overlooked Error

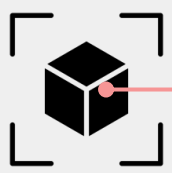
annotators missed the fire hydrant (class #10 in COCO) which the model detected with confidence 0.998, leading to a low Overlooked-score in ObjectLab.



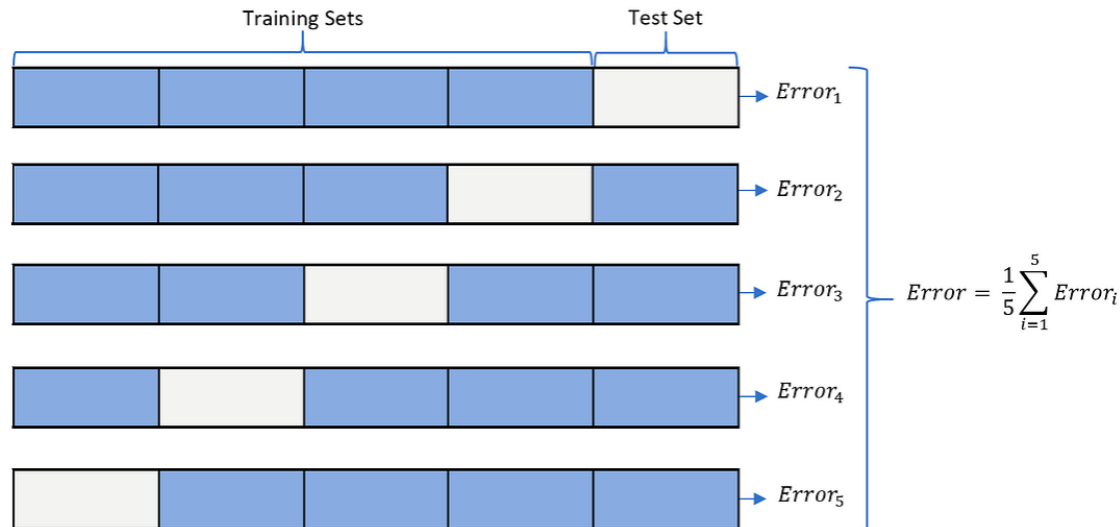
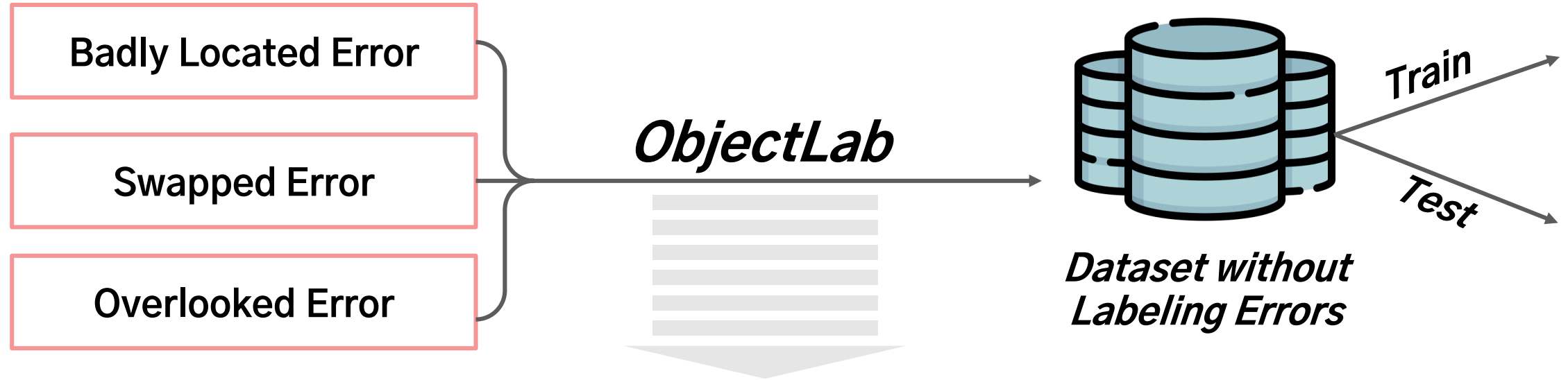
INTRO: Problem Statements



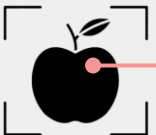
***You Do Not Need
to Change Your Models!***
→ *Just use any type of Detection Model*



INTRO: Problem Statements



: 5-Fold Cross-validation



Related Works: TIDE

→ A General Toolbox for Identifying Object Detection Errors

mAP

$$\text{Precision} = \frac{TP}{TP + FP}$$

TP = True positive

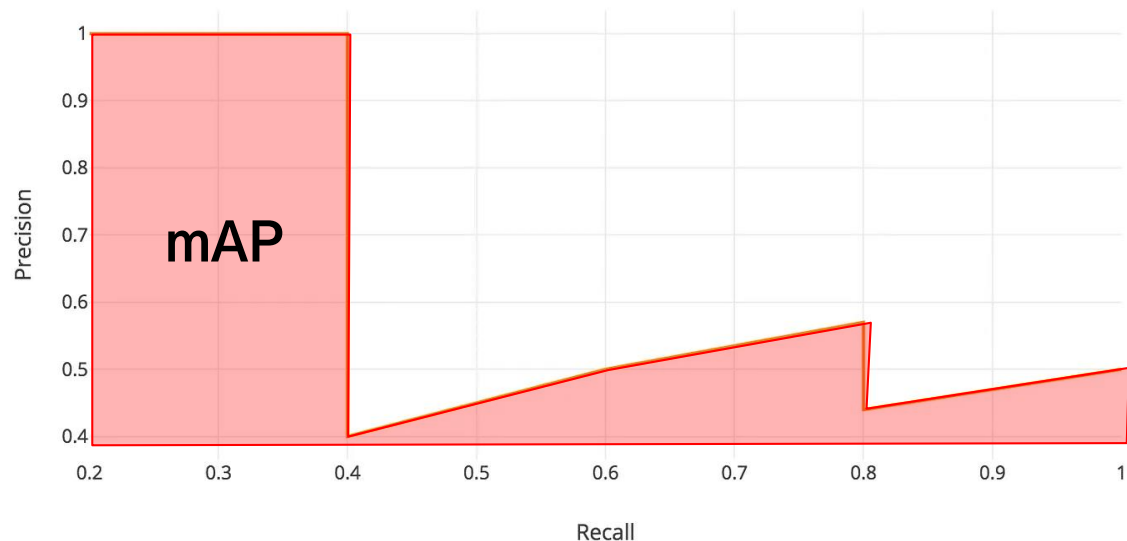
TN = True negative

FP = False positive

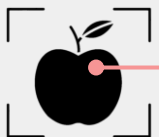
FN = False negative

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$



- 오류 유형이 서로 얽혀있어 각 오류 유형이 mAP에 얼마나 영향을 미치는지 측정하기 어려워, detector의 오류 분석에 활용하기 어려움
- mAP만을 최적화함으로써 application마다 다를 수 있는 오류 유형의 상대적 중요성을 생략할 수 있음 (ex. 종양 탐지에서 상자 위치파악보다 분류 정확도가 더 중요함)



Related Works: TIDE

→ A General Toolbox for Identifying Object Detection Errors

TIDE

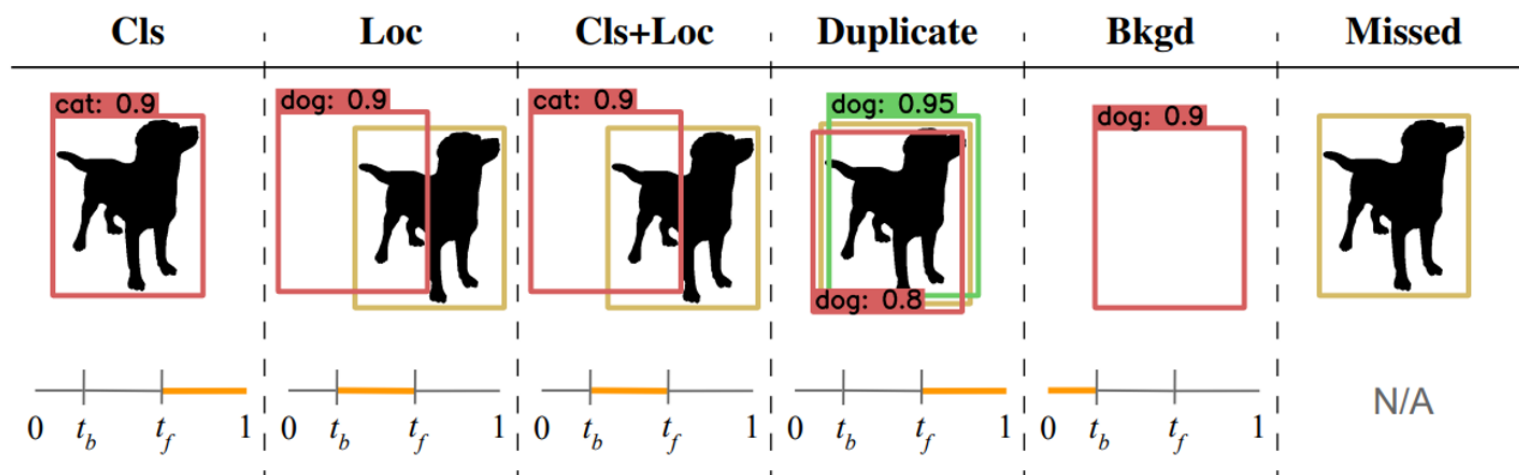


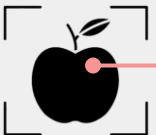
Fig. 1: **Error Type Definitions.** We define 6 error types, illustrated in the *top row*, where box colors are defined as: ■ = false positive detection; ■ = ground truth; ■ = true positive detection. The IoU with ground truth for each error type is indicated by an orange highlight and shown in the *bottom row*.

- Error를 6가지 유형으로 분류

- 각 오류의 기여도를 측정하여 오류 원인 분석 가능

- Contribution

- 오류 유형을 간결하게 요약하여 한 눈에 비교할 수 있음
- 결론에 영향을 미칠 수 있는 교란 변수가 없도록 각 오류 유형의 기여도를 완전히 분리함
- 오류의 원인을 구별하여 원하는 더 정밀한 분석이 가능함



Related Works: TIDE

→ A General Toolbox for Identifying Object Detection Errors

TIDE

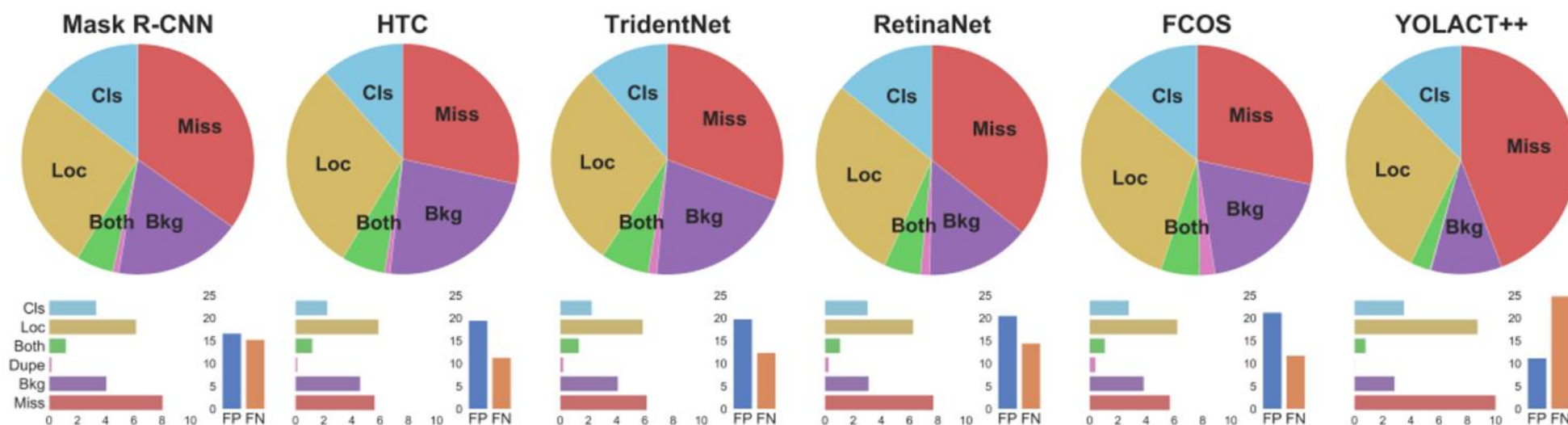
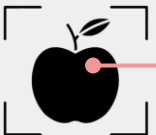


Fig. 3: Summary of errors on COCO Detection. Our model specific error analysis applied to various object detectors on COCO. The pie chart shows the relative contribution of each error, while the bar plots show their absolute contribution. For instance segmentation results, see the Appendix.



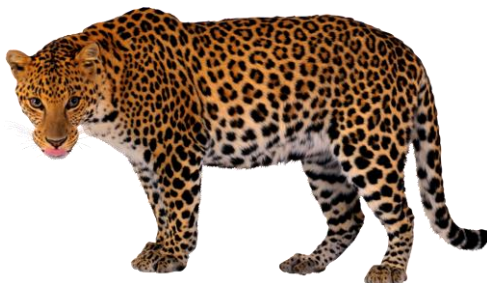
Related Works: Confident Learning Object Detection

Detecting Swapped Dataset

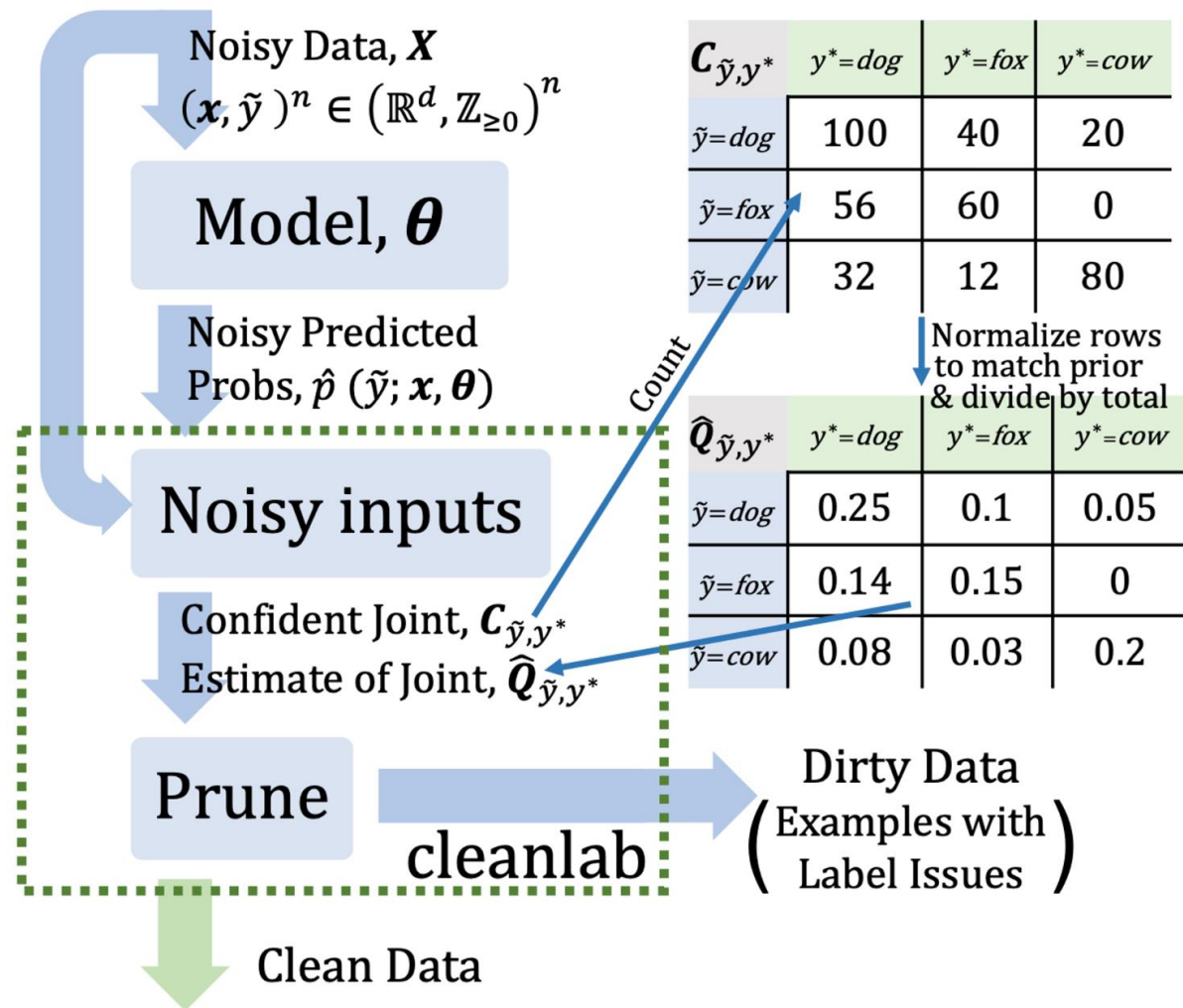
- **Assumption:** 결국 특정 클래스로 잘못 예측되는건 Prior latent vector가 얼마나 유사한지로 결정된다!

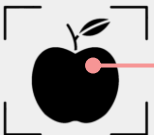


≈
confusing



≠
obvious





Related Works: Label Quality Score

Importance of Label Quality Scores

- **LED(Label Error Detection)**: 어떤 이미지가 잘못 라벨링 되는지를 식별하는 것
- Swin Transformer 모델을 confidence weighted entropy 나 self-confidence scores를 썼을 때 가장 결과가 좋았음.
- least-confidence와 entropy scores 는 성능이 제일 안좋았음.

Table 1. Lift @ 100 for LED with Swin Transformer model.

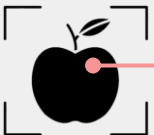
Label Quality Score	roman-numeral	food-101n	cifar-10n-worst	cifar-10n-agg	cifar-10s
Self Confidence	5.78	4.99	2.49	11.10	5.01
Normalized Margin	4.34	4.45	2.49	11.10	5.01
Adjusted Self Confidence	6.24	4.34	2.49	11.10	5.01
Adjusted Normalized Margin	4.34	4.07	2.49	11.10	5.01
Confidence Weighted Entropy	6.16	4.83	2.49	11.10	5.01
Entropy	4.11	5.37	1.84	5.77	0.10
Least Confidence	3.27	5.32	2.06	4.99	0.30

Table 2. Lift @ #Errors for LED with Swin Transformer model.

Label Quality Score	roman-numeral	food-101n	cifar-10n-worst	cifar-10n-agg	cifar-10s
Self Confidence	4.05	3.80	2.36	10.34	4.94
Normalized Margin	3.89	3.16	2.37	10.36	4.92
Adjusted Self Confidence	4.11	3.42	2.35	10.35	4.87
Adjusted Normalized Margin	3.85	3.33	2.38	10.37	4.92
Confidence Weighted Entropy	4.17	3.84	2.33	10.29	4.93
Entropy	3.01	2.81	1.28	3.59	0.52
Least Confidence	2.84	2.73	1.32	3.67	0.61

** Score가 높다 == Label Error를 잘 찾아냈다

Q & A

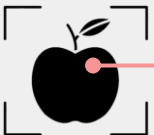


Methods: Notation

- I : Image
- B : Bounding Boxes
- $c(B) \in 1, \dots, K$: bbox 내부에 있는 object의 클래스
- $L(I)$: 주어진 label (데이터셋에서 주어진 GT)
- $\hat{s}(I)$: 이미지 I 의 **label quality score** -> score가 낮을수록 잘못 레이블됐을 가능성이 높다!
- $\hat{B}$: 모델이 예측한 Bounding Boxes
- $\hat{P}(I)$: 모델이 예측한 bbox output (이미지에서 예측한 전체 bbox들의 집합)
- $\hat{c}(\hat{B})$: 모델이 예측한 bbox 내부 object의 클래스
- $\hat{p}(\hat{B})$: $\hat{B}$ 가 클래스 $\hat{c}(\hat{B})$ 에 해당할 **confidence 값**
- τ : 모델의 confidence($\hat{p}$)에 대한 **threshold 값**



Legend	
■	given label
■	predicted label
classes	
■	sports ball
■	tennis racket
0	person



Methods: ObjectLab Algorithm

ObjectLab의 Label Score $\hat{s}$

Algorithm 1 ObjectLab score $\hat{s}(I)$ for an image I

Require: given label $\mathcal{L}(I)$, model prediction $\hat{\mathcal{P}}(I)$

1: $q_1, \dots, q_N \leftarrow \text{BadlocBoxScores}(\mathcal{L}(I), \hat{\mathcal{P}}(I))$

2: $\hat{s}_{badloc}(I) \leftarrow \text{softmin}(q_1, \dots, q_N)$

3: $q_1, \dots, q_N \leftarrow \text{SwappedBoxScores}(\mathcal{L}(I), \hat{\mathcal{P}}(I))$

4: $\hat{s}_{swap}(I) \leftarrow \text{softmin}(q_1, \dots, q_N)$

5: $q_1, \dots, q_M \leftarrow \text{OverlookedBoxScores}(\mathcal{L}(I), \hat{\mathcal{P}}(I))$

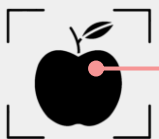
6: $\hat{s}_{overlook}(I) \leftarrow \text{softmin}(q_1, \dots, q_M)$

return Geometric mean of $\hat{s}_{badloc}$, $\hat{s}_{swap}$, $\hat{s}_{overlook}$ for I

- I : Image
- B : Bounding Boxes
- $c(B) \in 1, \dots, K$: bbox 내부에 있는 object의 클래스
- $L(I)$: 주어진 label (데이터셋에서 주어진 GT)
- $\hat{s}(I)$: 이미지 I 의 **label quality score**
- $\hat{B}$: 모델이 예측한 Bounding Boxes
- $\hat{\mathcal{P}}(I)$: 모델이 예측한 bbox output
- $\hat{c}(\hat{B})$: 모델이 예측한 bbox 내부 object의 클래스
- $\hat{p}(\hat{B})$: $\hat{B}$ 가 클래스 $\hat{c}(\hat{B})$ 에 해당할 **confidence** 값
- τ : 모델의 confidence($\hat{p}$)에 대한 **threshold** 값

$$= \left(\hat{S}_{badloc} \times \hat{S}_{swap} \times \hat{S}_{overlook} \right)^{1/3}$$

$\hat{S}_{badloc}$: GT의 bbox 위치가 정확하지 않은 error에 대한 score
 $\hat{S}_{swap}$: GT의 bbox 위치는 맞지만, Class가 틀린 경우에 대한 score
 $\hat{S}_{overlook}$: GT의 bbox가 존재하지 않는 경우에 대한 score

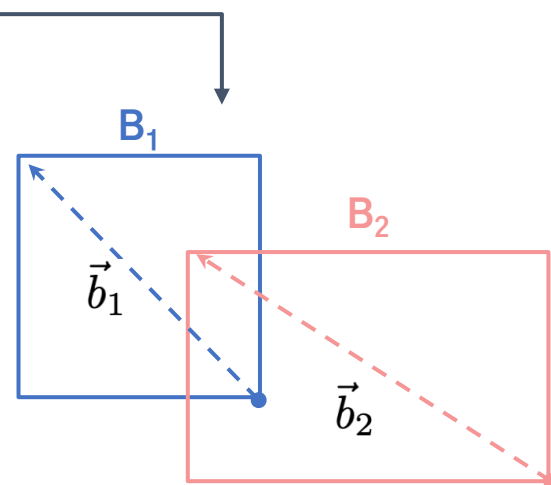


Methods: Similarity Function

: 한 이미지에서 나온 bbox pair들에 대해서 Similarity를 계산할 수 있는 식

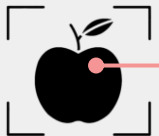
$$\text{sim}(B_1, B_2) = \alpha \cdot k(B_1, B_2) + (1 - \alpha) \cdot \text{IoU}(B_1, B_2)$$

- $k(,)$: 두 박스의 꼭짓점 정보를 가지고 있는 4D vector에 대한 Gaussian kernel similarity
- Sim_* : annotated bbox와 모델의 predicted bbox pair간의 minimum possible similarity
- $q^* = 1$: maximum possible quality estimate for any box (across all mistake subtypes)



$$\exp(-\|\vec{b}_1 - \vec{b}_2\|/\sigma)$$

$$\alpha = \sigma = 0.1$$



Methods: Similarity Function

: 한 이미지에서 나온 bbox pair들에 대해서 Similarity를 계산할 수 있는 식

$$sim(B_1, B_2) = \alpha \cdot k(B_1, B_2) + (1 - \alpha) \cdot IoU(B_1, B_2)$$

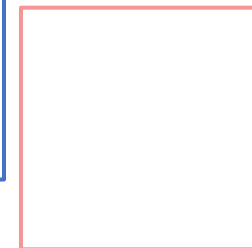
- $k(,)$: 두 박스의 꼭짓점 정보를 가지고 있는 4D vector에 대한 Gaussian kernel similarity
- Sim_* : annotated bbox와 모델의 predicted bbox pair간의 **minimum possible similarity**
- $q^* = 1$: **maximum possible quality estimate** for any box (across all mistake subtypes)

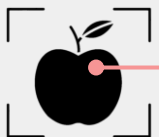
if in case of
badly located error,

B_{any}



B_{any}





Methods: Badly Located Box Scores

$$\hat{s} = \left(\hat{S}_{badloc} \times \hat{S}_{swap} \times \hat{S}_{overlook} \right)^{1/3}$$

$\hat{S}_{badloc}$: GT의 bbox 위치가 정확하지 않은 error에 대한 score
 $\hat{S}_{swap}$: GT의 bbox 위치는 맞지만, Class가 틀린 경우에 대한 score
 $\hat{S}_{overlook}$: GT의 bbox가 존재하지 않는 경우에 대한 score

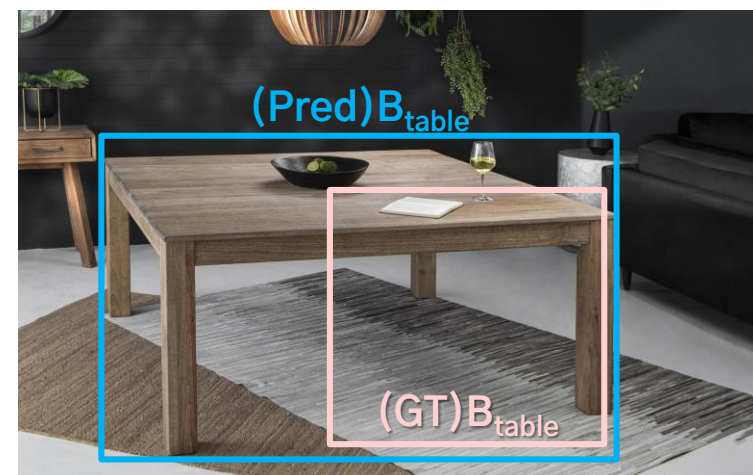
- I : Image
- B : Bounding Boxes
- $c(B) \in 1, \dots, K$: bbox 내부에 있는 object의 클래스
- $L(I)$: 주어진 label (데이터셋에서 주어진 GT)
- $\hat{s}(I)$: 이미지 I 의 label quality score
- $\hat{B}$: 모델이 예측한 Bounding Boxes
- $\hat{P}(I)$: 모델이 예측한 bbox output
- $\hat{c}(\hat{B})$: 모델이 예측한 bbox 내부 object의 클래스
- $\hat{p}(\hat{B})$: $\hat{B}$ 가 클래스 $\hat{c}(\hat{B})$ 에 해당할 confidence 값
- τ : 모델의 confidence($\hat{p}$)에 대한 threshold 값

Algorithm 2 BadlocBoxScores for image I

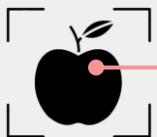
Require: given label $\mathcal{L}(I)$, model prediction $\hat{P}(I)$

- 1: $\text{scores} \leftarrow \{\}$
- 2: **for** annotated box $B \in \mathcal{L}(I)$ **do**
- 3: Let $k = c(B)$ denote its annotated class and $\mathcal{P}_k := \{\hat{B} \in \hat{P}(I) : \hat{c}(\hat{B}) = k, IoU(\hat{B}, B) > 0\}$ denote the predicted boxes with the same predicted class that overlap with B .
- 4: $\text{scores.append}(q)$ where $q \leftarrow q^*$ **if** $\mathcal{P}_k = \emptyset$ **else:** $q \leftarrow \max_{\hat{B} \in \mathcal{P}_k} sim(B, \hat{B})$

return scores



$$sim(B_1, B_2) = \alpha \cdot k(B_1, B_2) + (1 - \alpha) \cdot IoU(B_1, B_2)$$



Methods: Softmin Pooling

$$\hat{\mathbf{s}} = \left(\hat{S}_{badloc} \times \hat{S}_{swap} \times \hat{S}_{overlook} \right)^{1/3}$$

$\hat{S}_{badloc}$: GT의 bbox 위치가 정확하지 않은 error에 대한 score
 $\hat{S}_{swap}$: GT의 bbox 위치는 맞지만, Class가 틀린 경우에 대한 score
 $\hat{S}_{overlook}$: GT의 bbox가 존재하지 않는 경우에 대한 score

- I : Image
- B : Bounding Boxes
- $c(B) \in 1, \dots, K$: bbox 내부에 있는 object의 클래스
- $L(I)$: 주어진 label (데이터셋에서 주어진 GT)
- $\hat{\mathbf{s}}(I)$: 이미지 I 의 label quality score
- $\hat{B}$: 모델이 예측한 Bounding Boxes
- $\hat{P}(I)$: 모델이 예측한 bbox output
- $\hat{c}(\hat{B})$: 모델이 예측한 bbox 내부 object의 클래스
- $\hat{p}(\hat{B})$: $\hat{B}$ 가 클래스 $\hat{c}(\hat{B})$ 에 해당할 confidence 값
- τ : 모델의 confidence($\hat{p}$)에 대한 threshold 값

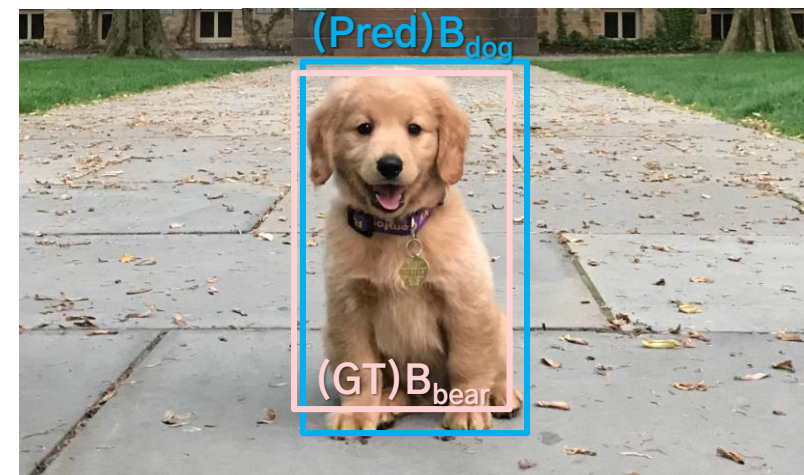
Algorithm 3 SwappedBoxScores for image I

Require: given label $\mathcal{L}(I)$, model prediction $\hat{\mathcal{P}}(I)$

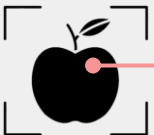
- 1: $\text{scores} \leftarrow \{\}$
- 2: **for** annotated box $B \in \mathcal{L}(I)$ **do**
- 3: Let $k = c(B)$ denote its annotated class and $\mathcal{P}_{-k} := \{\hat{B} \in \hat{\mathcal{P}}(I) : \hat{c}(\hat{B}) \neq k, \hat{p}(\hat{B}) > \tau_{\uparrow}\}$ be the predicted boxes with another predicted class whose confidence exceeds threshold $\tau_{\uparrow}$.
- 4: $\text{scores.append}(q)$ where $q \leftarrow q^*$ **if** $\mathcal{P}_{-k} = \emptyset$ **else:** $q \leftarrow 1 - \max_{\hat{B} \in \mathcal{P}_{-k}} \text{sim}(B, \hat{B})$

return scores

$$\tau_{\uparrow} = 0.95$$



$$\text{sim}(B_1, B_2) = \alpha \cdot k(B_1, B_2) + (1 - \alpha) \cdot \text{IoU}(B_1, B_2)$$



Methods: Softmin Pooling

$$\hat{\mathcal{S}} = \left(\hat{\mathcal{S}}_{badloc} \times \hat{\mathcal{S}}_{swap} \times \hat{\mathcal{S}}_{overlook} \right)^{1/3}$$

: GT의 bbox 위치가 정확하지 않은
error에 대한 score

: GT의 bbox 위치는 맞지만,
Class가 틀린 경우에 대한 score

: GT의 bbox가 존재하지 않는
경우에 대한 score

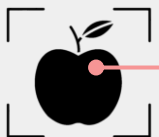
- I : Image
- B : Bounding Boxes
- $c(B) \in 1, \dots, K$: bbox 내부에 있는 object의 클래스
- $L(I)$: 주어진 label (데이터셋에서 주어진 GT)
- $\hat{\mathcal{S}}(I)$: 이미지 I 의 label quality score
- $\hat{B}$: 모델이 예측한 Bounding Boxes
- $\hat{\mathcal{P}}(I)$: 모델이 예측한 bbox output
- $\hat{c}(\hat{B})$: 모델이 예측한 bbox 내부 object의 클래스
- $\hat{p}(\hat{B})$: $\hat{B}$ 가 클래스 $\hat{c}(\hat{B})$ 에 해당할 confidence 값
- τ : 모델의 confidence($\hat{p}$)에 대한 threshold 값

Algorithm 4 OverlookedBoxScores for image I

Require: given label $\mathcal{L}(I)$, model prediction $\hat{\mathcal{P}}(I)$

- 1: $\text{scores} \leftarrow \{\}$;
- 2: **for** predicted box $\hat{B} \in \hat{\mathcal{P}}(I)$ with $\hat{p}(\hat{B}) > \tau_{\uparrow}$ **do** $\tau_{\uparrow} = 0.95$
- 3: Let $k = \hat{c}(\hat{B})$ denote its predicted class and
 $\mathcal{L}_k := \{B \in \mathcal{L}(I) : c(B) = k, IoU(B, \hat{B}) = 0\}$
denote the subset of annotated boxes for
class k that do not overlap with $\hat{B}$.
- 4: **if** $\mathcal{L}_k = \emptyset$ **then**
- 5: $q = \text{sim}_* \cdot (1 - \hat{p}(\hat{B}))$
- 6: **else**
- 7: $q = \max_{B \in \mathcal{L}_k} \text{sim}(B, \hat{B})$
- 8: $\text{scores.append}(q)$
- 9: **if** $\text{scores} = \emptyset$: $\text{scores} = \{q^*\}$
return scores





Methods: Softmin Pooling

$$\hat{s} = \left(\hat{S}_{badloc} \times \hat{S}_{swap} \times \hat{S}_{overlook} \right)^{1/3}$$

$\hat{S}_{badloc}$: GT의 bbox 위치가 정확하지 않은 error에 대한 score
 $\hat{S}_{swap}$: GT의 bbox 위치는 맞지만, Class가 틀린 경우에 대한 score
 $\hat{S}_{overlook}$: GT의 bbox가 존재하지 않는 경우에 대한 score

★★ 정확히 스코어 뭘로 잘랐는지?

- I : Image
- B : Bounding Boxes
- $c(B) \in 1, \dots, K$: bbox 내부에 있는 object의 클래스
- $L(I)$: 주어진 label (데이터셋에서 주어진 GT)
- $\hat{s}(I)$: 이미지 I의 label quality score
- $\hat{B}$: 모델이 예측한 Bounding Boxes
- $\hat{P}(I)$: 모델이 예측한 bbox output
- $\hat{c}(\hat{B})$: 모델이 예측한 bbox 내부 object의 클래스
- $\hat{p}(\hat{B})$: $\hat{B}$ 가 클래스 $\hat{c}(\hat{B})$ 에 해당할 confidence 값
- τ : 모델의 confidence($\hat{p}$)에 대한 threshold 값

Algorithm 1 ObjectLab score $\hat{s}(I)$ for an image I

Require: given label $\mathcal{L}(I)$, model prediction $\hat{\mathcal{P}}(I)$

1: $q_1, \dots, q_N \leftarrow \text{BadlocBoxScores}(\mathcal{L}(I), \hat{\mathcal{P}}(I))$

2: $\hat{s}_{badloc}(I) \leftarrow \text{softmin}(q_1, \dots, q_N)$

3: $q_1, \dots, q_N \leftarrow \text{SwappedBoxScores}(\mathcal{L}(I), \hat{\mathcal{P}}(I))$

4: $\hat{s}_{swap}(I) \leftarrow \text{softmin}(q_1, \dots, q_N)$

5: $q_1, \dots, q_M \leftarrow \text{OverlookedBoxScores}(\mathcal{L}(I), \hat{\mathcal{P}}(I))$

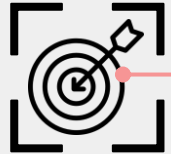
6: $\hat{s}_{overlook}(I) \leftarrow \text{softmin}(q_1, \dots, q_M)$

return Geometric mean of $\hat{s}_{badloc}, \hat{s}_{swap}, \hat{s}_{overlook}$ for I

Softmin

$$\bar{q} = \langle \vec{q}, \text{softmax}(1 - \vec{q}) \rangle$$

Q & A



Experiments: Dataset and Models

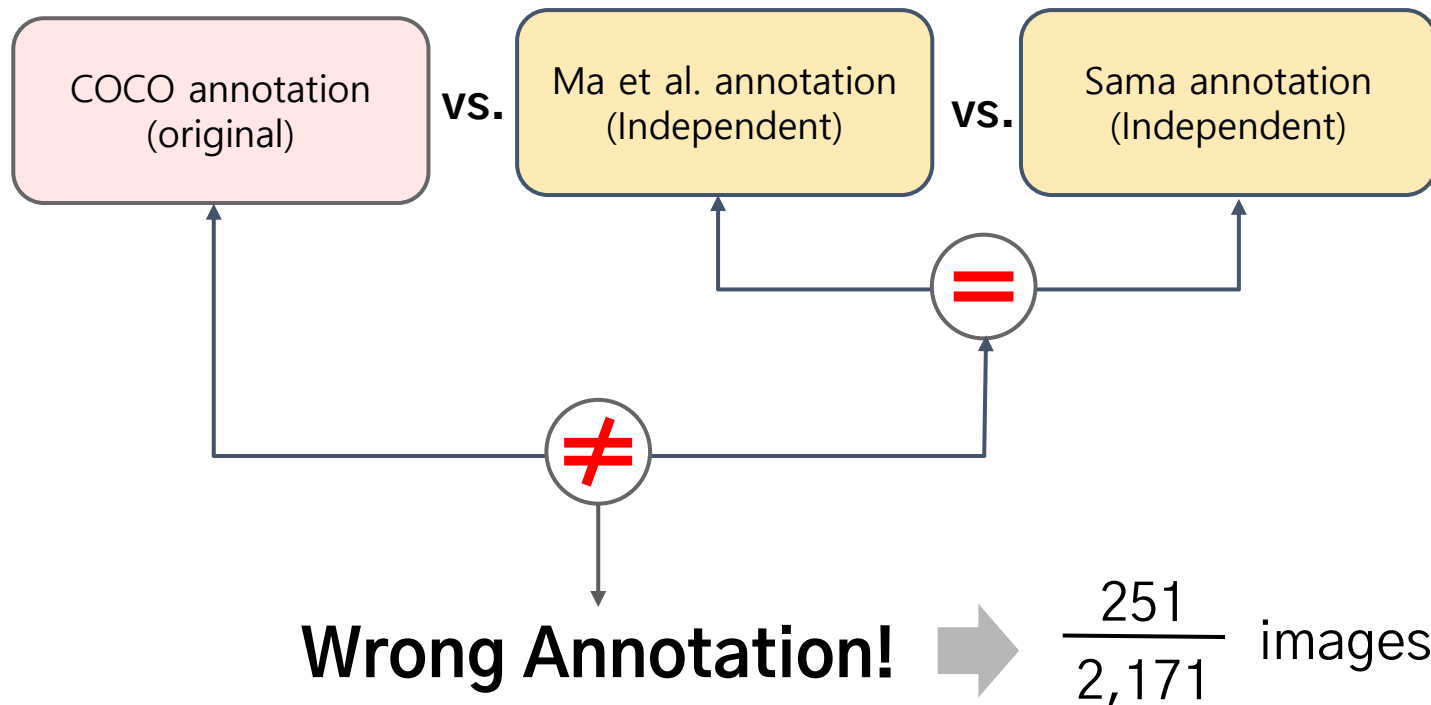
COCO-bench Dataset

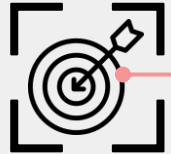
5 Classes: $\{person, chair, cup, car, traffic\ light\}$



Figure 2. Example of a naturally mislabeled image in the COCO-bench dataset that receives low ObjectLab score. We show the original dataset label, which contains no boxes beyond those depicted for **Car** (class #0). Here the seated **person** and the **chair** were overlooked in the given label, even though these are among the 5 COCO-bench classes.

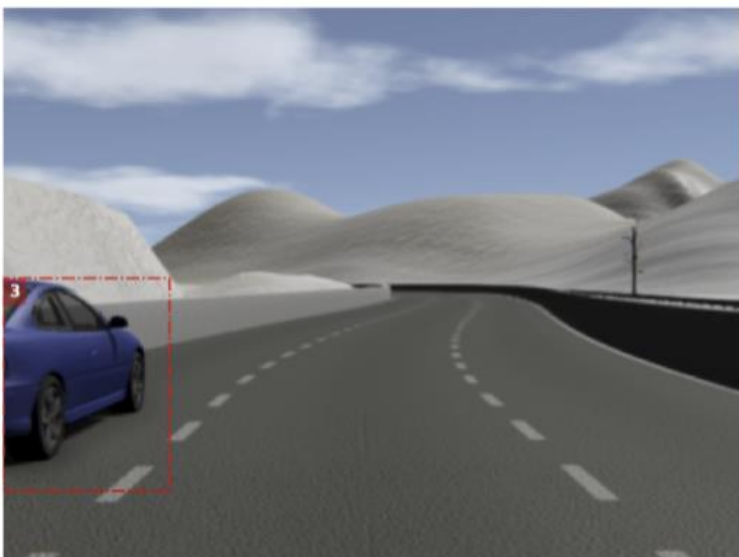
Compares



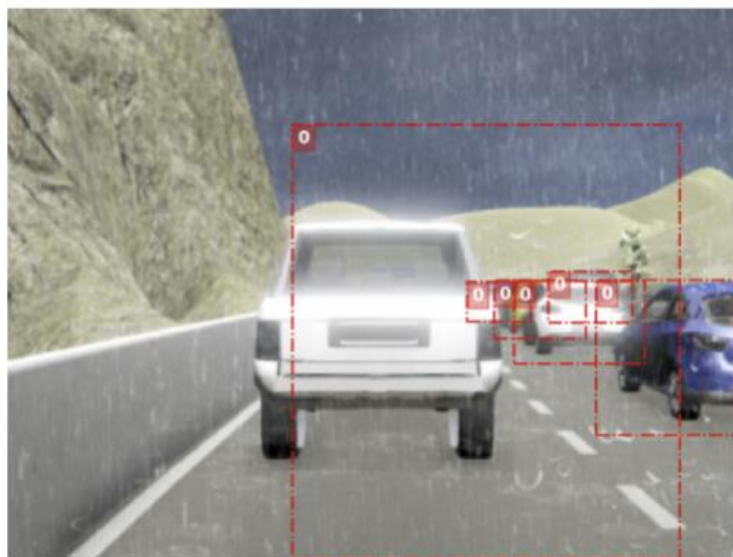


Experiments: Dataset and Models

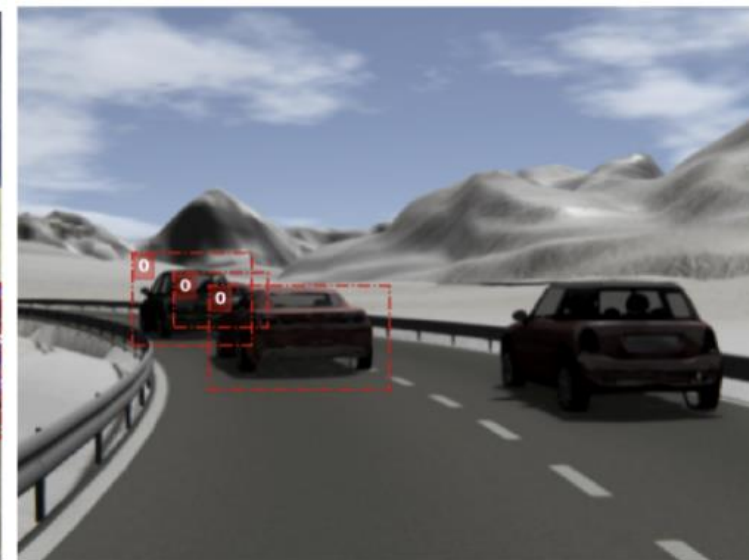
SYNTHIA-AL Dataset



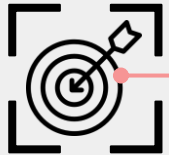
Car(#0)인데 Bicycle(#3)라고
잘못 라벨링되어 있음



가운데 Car(#0)의 BBox 위치가
정확하지 않음



마지막 Car의 BBox가
missing되어 있음



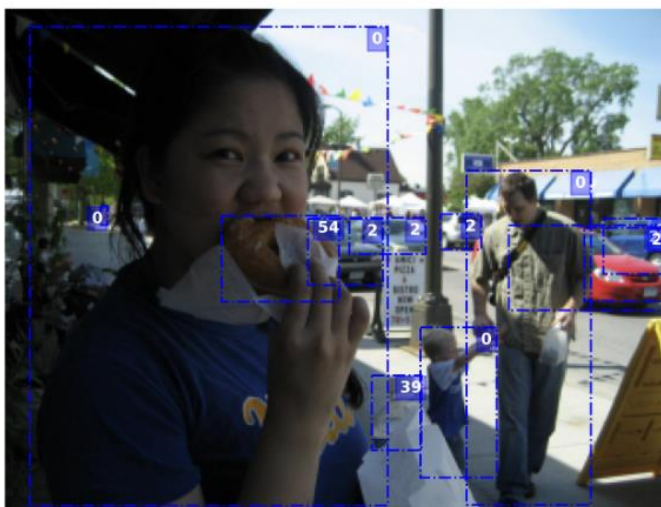
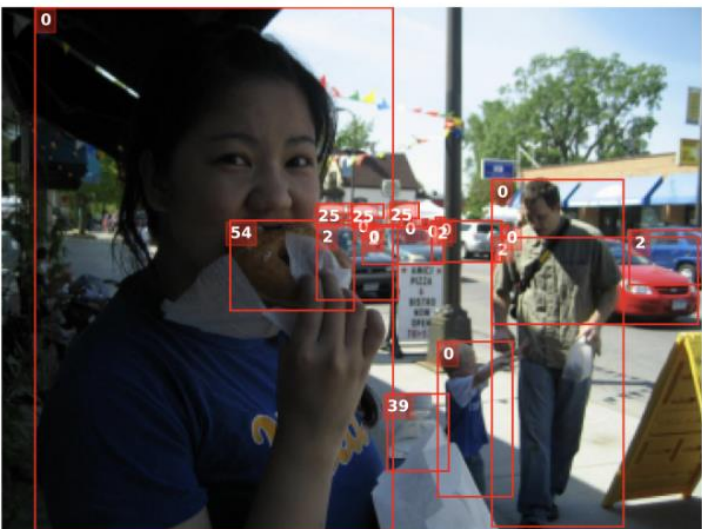
Experiments: Dataset and Models

COCO-full Dataset : Badly Located Error



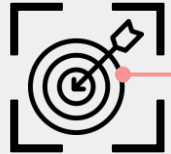
Badly Located
of **Train** BBox

Legend	
■	given label
■	predicted label
classes	
0	person
6	train
9	traffic light



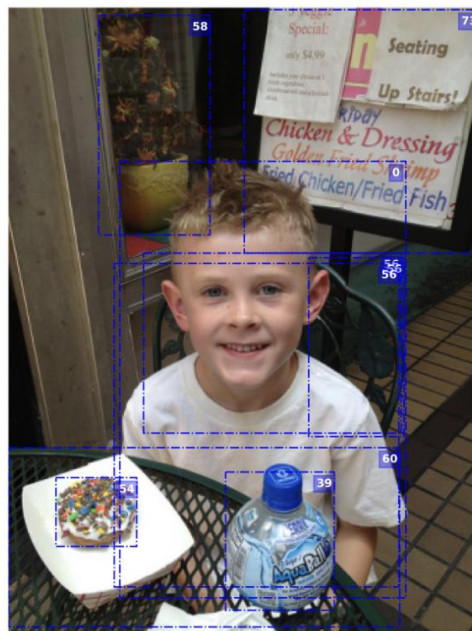
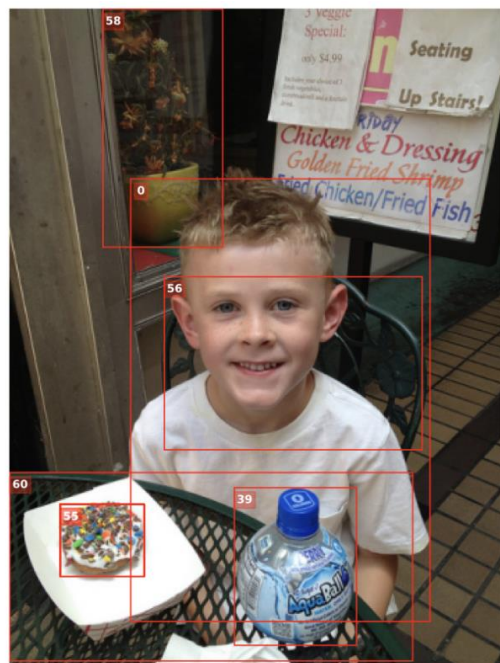
Badly Located
of **Person** BBox

Legend	
■	given label
■	predicted label
classes	
25	umbrella
2	car
0	person
54	donut
39	bottle

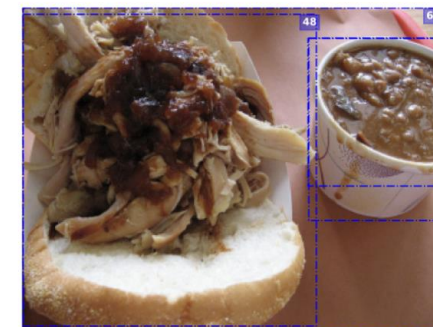
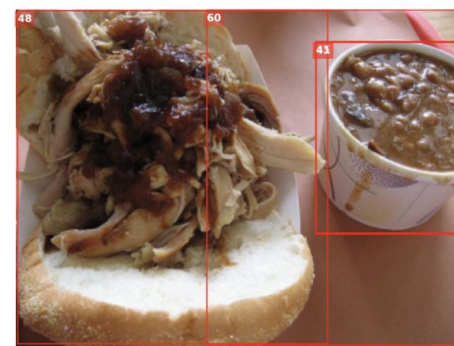


Experiments: Dataset and Models

COCO-full Dataset : Swapped Error



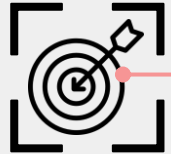
Legend
■ given label
■ predicted label
classes
73 book
58 cake
58 potted plant
60 dining table
0 person
56 chair
54 donut
39 bottle



Legend
■ given label
■ predicted label
classes
41 cup
60 dining table
48 sandwich
43 bowl

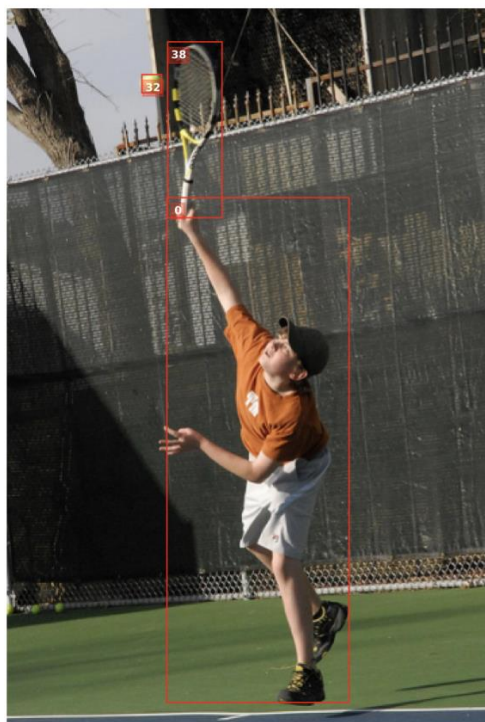
Swapped between
Cake $\leftrightarrow$ Donut

Swapped between
Bowl $\leftrightarrow$ Cup



Experiments: Dataset and Models

COCO-full Dataset : Overlooked Error



Legend
■ given label
■ predicted label
classes
■ sports ball
■ tennis racket
0 person

BBoxes of **Sports Balls** are Overlooked



BBox of a **Person** is Overlooked

Legend
■ given label
■ predicted label
classes
0 person

Experiments: Metrics

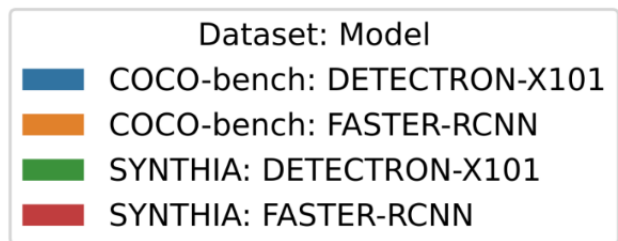
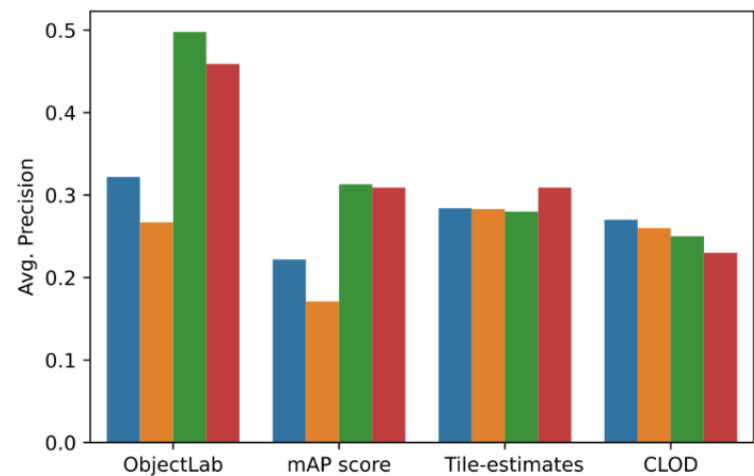
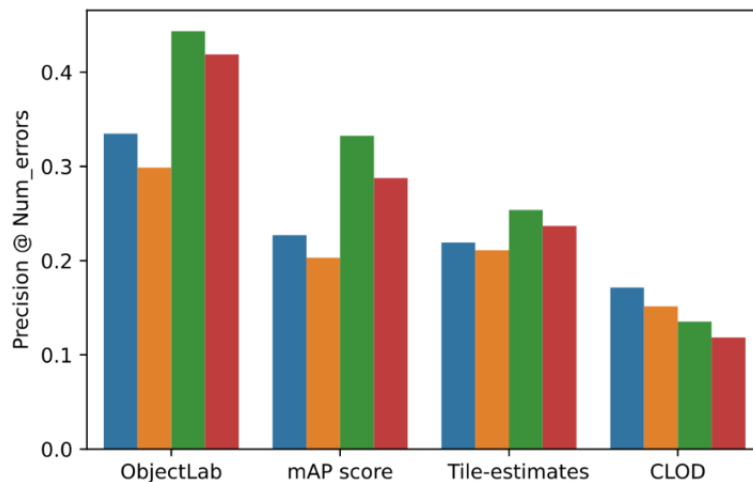
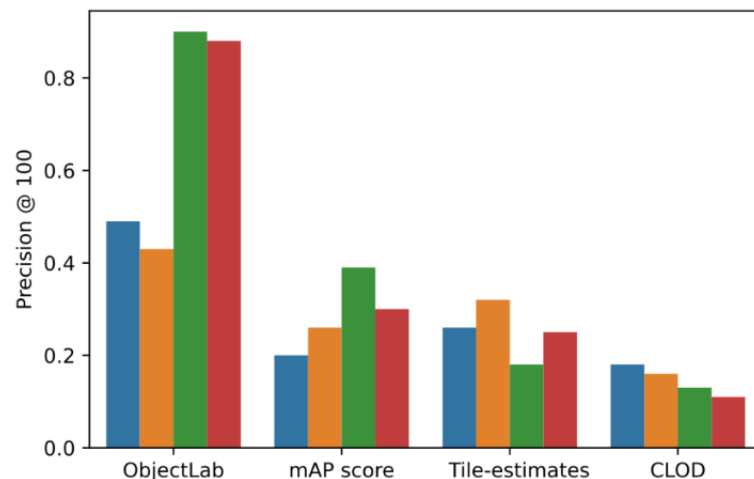


Table 1. Precision@100 achieved by various label quality scores for detecting mislabeled images in the COCO-full dataset.

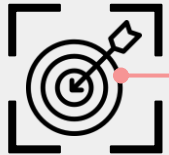
MODEL	OBJECTLAB	MAP
DETECTRON-X101	0.71	0.22
FASTER-RCNN	0.57	0.17



// ObjectLab results, we estimate that in **COCO 2017** around:

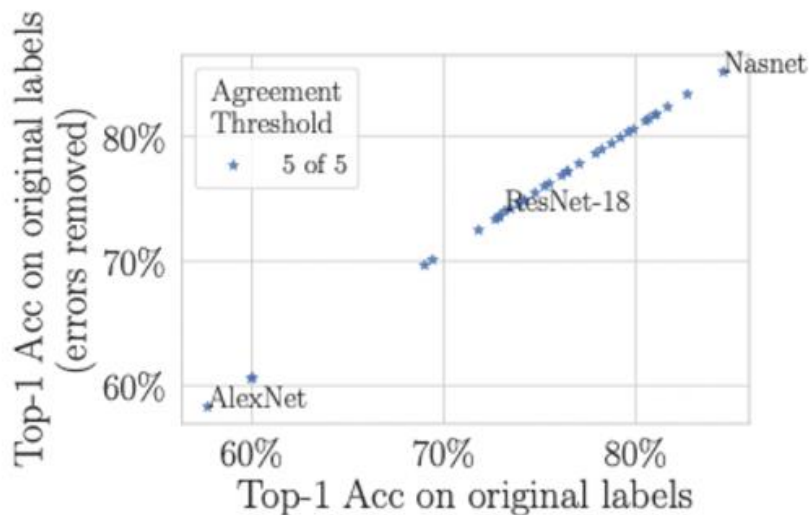
3% have a *Badly Located* error,
0.7% have a *Swapped* error,
and 5% of images have an *Overlooked* error.

Figure 5. Evaluating various label quality scoring methods across two models and two datasets where ground truth label errors are known.

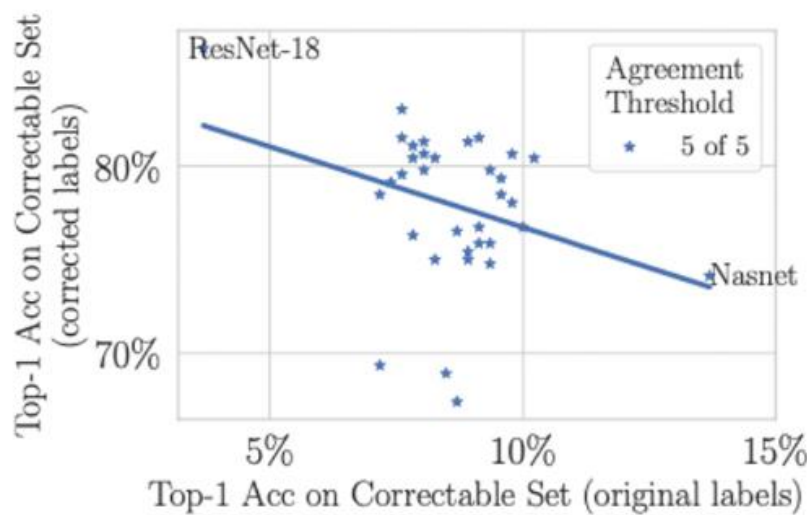


References: Label Errors in Test Dataset

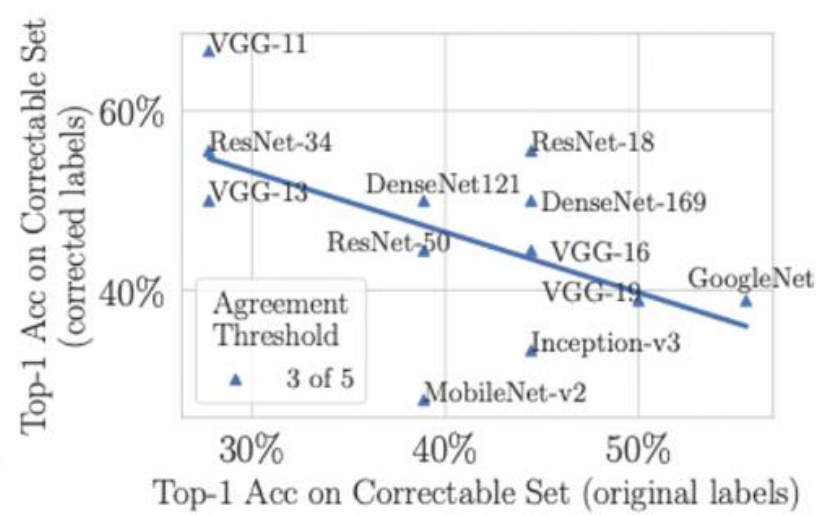
Detecting Swapped Dataset : 큰 모델일수록 원래 테스트셋에서는 높은 성능이지만 고친 데이터에 대해서 떨어짐



(a) ImageNet val set acc.



(b) ImageNet correctable set acc.



(c) CIFAR-10 correctable set acc.

Implications of label errors in test data

1. 작은 모델일수록 보이지 않는 regularization 이점을 확인함 (작은 모델일 경우 고친 데이터에 대해서 성능 올라감)
2. 큰 모델은 system 자체의 label error의 패턴을 학습하여 좋은 성능을 가져온다.



References: The Effect of Improving Annotation Quality

Old

Dataset에서 주어진 Annotation

New

Annotation Error를 올바르게 고친 버전

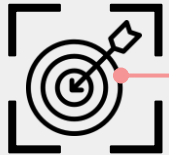
Method	train/test	mAP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L	car	chair	cup	person	traffic light
Faster RCNN [18]	old/old	0.352	0.591	0.365	0.207	0.431	0.501	0.399	0.216	0.377	0.516	0.251
	new/old	0.338	0.573	0.347	0.198	0.414	0.497	0.390	0.180	0.366	0.515	0.239
	old/new	0.309	0.514	0.321	0.163	0.414	0.501	0.335	0.203	0.363	0.376	0.268
	new/new	0.327	0.543	0.338	0.188	0.424	0.5	0.346	0.206	0.363	0.424	0.295
SSD [13]	old/old	0.16	0.335	0.136	0.033	0.193	0.356	0.179	0.091	0.145	0.316	0.068
	new/old	0.148	0.315	0.125	0.029	0.177	0.34	0.168	0.073	0.143	0.301	0.055
	old/new	0.132	0.279	0.113	0.024	0.184	0.384	0.149	0.098	0.134	0.208	0.071
	new/new	0.126	0.275	0.103	0.024	0.172	0.34	0.141	0.092	0.133	0.206	0.062
YOLOv3 [17]	old/old	0.249	0.5	0.22	0.117	0.322	0.395	0.261	0.157	0.261	0.422	0.146
	new/old	0.229	0.467	0.197	0.108	0.292	0.371	0.258	0.124	0.234	0.409	0.117
	old/new	0.208	0.425	0.186	0.081	0.305	0.386	0.217	0.137	0.239	0.290	0.157
	new/new	0.206	0.431	0.175	0.094	0.29	0.371	0.214	0.142	0.213	0.318	0.141
EfficientDet [23]	old/old	0.14	0.266	0.132	0.043	0.186	0.261	0.148	0.046	0.132	0.325	0.051
	new/old	0.138	0.26	0.13	0.04	0.18	0.267	0.128	0.056	0.102	0.23	0.054
	old/new	0.114	0.219	0.102	0.031	0.168	0.261	0.122	0.052	0.121	0.219	0.053
	new/new	0.114	0.219	0.104	0.032	0.171	0.265	0.151	0.048	0.109	0.327	0.056

* All values are related to mAP except for AP_{50} and AP_{75} . The results of DETR [5] were very low (mAP<0.1) thus not included.

Table 2. The experimental results on COCO

{TRAIN} / {TEST}

- (old/old)일 때 좋은 경우가 많음



References: The Effect of Improving Annotation Quality

Old

Dataset에서 주어진 Annotation

New

Annotation Error를 올바르게 고친 버전

Method	train/test	mAP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L	building	car	dog	flower	person
Faster RCNN [18]	old/old	0.387	0.548	0.429	0.062	0.099	0.443	0.311	0.542	0.669	0.238	0.176
	new/old	0.349	0.494	0.387	0.091	0.086	0.393	0.219	0.507	0.666	0.223	0.131
	old/new	0.426	0.625	0.465	0.062	0.187	0.501	0.163	0.604	0.689	0.276	0.398
	new/new	0.465	0.664	0.501	0.072	0.179	0.54	0.226	0.665	0.692	0.282	0.459
SSD [13]	old/old	0.366	0.527	0.404	0.005	0.047	0.413	0.306	0.513	0.669	0.242	0.101
	new/old	0.332	0.485	0.371	0.002	0.045	0.372	0.225	0.483	0.666	0.209	0.078
	old/new	0.390	0.599	0.427	0.001	0.081	0.461	0.139	0.575	0.678	0.242	0.313
	new/new	0.403	0.618	0.442	0.002	0.085	0.478	0.172	0.602	0.684	0.228	0.328
YOLOv3 [17]	old/old	0.344	0.53	0.385	0.093	0.079	0.39	0.251	0.502	0.609	0.209	0.15
	new/old	0.312	0.484	0.351	0.09	0.076	0.351	0.154	0.491	0.588	0.191	0.139
	old/new	0.390	0.606	0.435	0.058	0.21	0.456	0.110	0.567	0.652	0.285	0.342
	new/new	0.417	0.64	0.46	0.074	0.194	0.482	0.172	0.627	0.631	0.269	0.402
EfficientDet [23]	old/old	0.413	0.553	0.452	0.057	0.082	0.464	0.334	0.533	0.691	0.286	0.125
	new/old	0.381	0.503	0.403	0.015	0.082	0.43	0.185	0.533	0.757	0.323	0.107
	old/new	0.439	0.608	0.478	0.091	0.147	0.509	0.145	0.603	0.754	0.274	0.416
	new/new	0.465	0.618	0.503	0.025	0.162	0.534	0.141	0.667	0.787	0.271	0.461
DETR [5]	old/old	0.379	0.515	0.397	0.038	0.045	0.436	0.367	0.463	0.751	0.205	0.108
	new/old	0.335	0.45	0.356	0.012	0.049	0.385	0.205	0.428	0.769	0.262	0.101
	old/new	0.394	0.592	0.407	0.012	0.142	0.479	0.133	0.525	0.772	0.167	0.352
	new/new	0.429	0.629	0.44	0.007	0.135	0.514	0.152	0.574	0.799	0.220	0.399

{TRAIN} / {TEST}

● (new/new)일 때 좋은 경우가 많음

* All values are related to mAP except for AP_{50} and AP_{75} .

Table 3. The experimental results on Open Images



Conclusions

1.

ObjectLab은 모델 구조 변화 없이 **Annotation Error**를 탐지하고, 이를 **올바르게 고쳐줄 수 있는** General한 Toolkit임

2.

Noisy Dataset으로 학습을 잘 시키는 방법에 대한 연구도 있지만, **데이터셋의 오류를 교정하여 좋은 데이터셋으로 학습 혹은 테스트를 해보자는 접근 방법임**

3.

데이터셋에 존재하는 약간의 에러는 너무 쉬운 Task가 되지 않도록 도와 모델의 Robustness를 올려줄 수 있으나, 에러가 많은 경우 학습에 방해가 됨

4.

Third-party Data Annotation Vendor에 의해 7%~80%의 레이블 에러 발생
→ 직접 데이터를 만들어야 하는 경우 유용하게 쓸 수 있을 것으로 보임

Q & A